Discussion 11:

Introduction to Parallel Computing and CUDA

Markus Hohle

University California, Berkeley

Machine Learning Algorithms

MSSE 277B, 3 Units

Spring 2025

"What was initially thought to be a simple process is in fact an incredibly complicated, intricate, and complex system that I've codified and organized into a few easy-to-follow rules that are more difficult to implement than you'd think."

Outline

- **Parallel Processing**
- **Using Map**
- **Using Process**
- **CUDA & PyTorch**

"What was initially thought to be a simple process is in fact an incredibly complicated, intricate, and complex system that I've codified and organized into a few easy-to-follow rules that are more difficult to implement than you'd think."

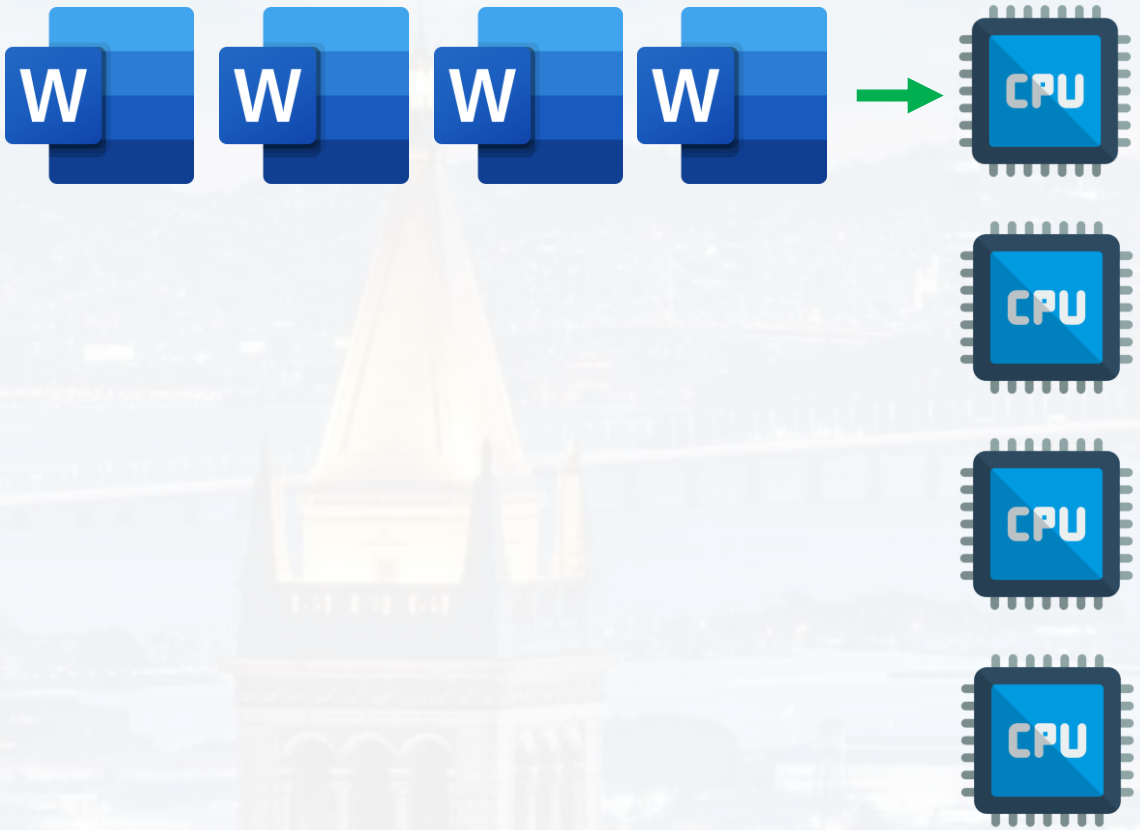
Outline

- **Parallel Processing**
- Using Map
- Using Process
- CUDA & PyTorch

Parallel & CUDA:

even for moderate models and data: → computational limits

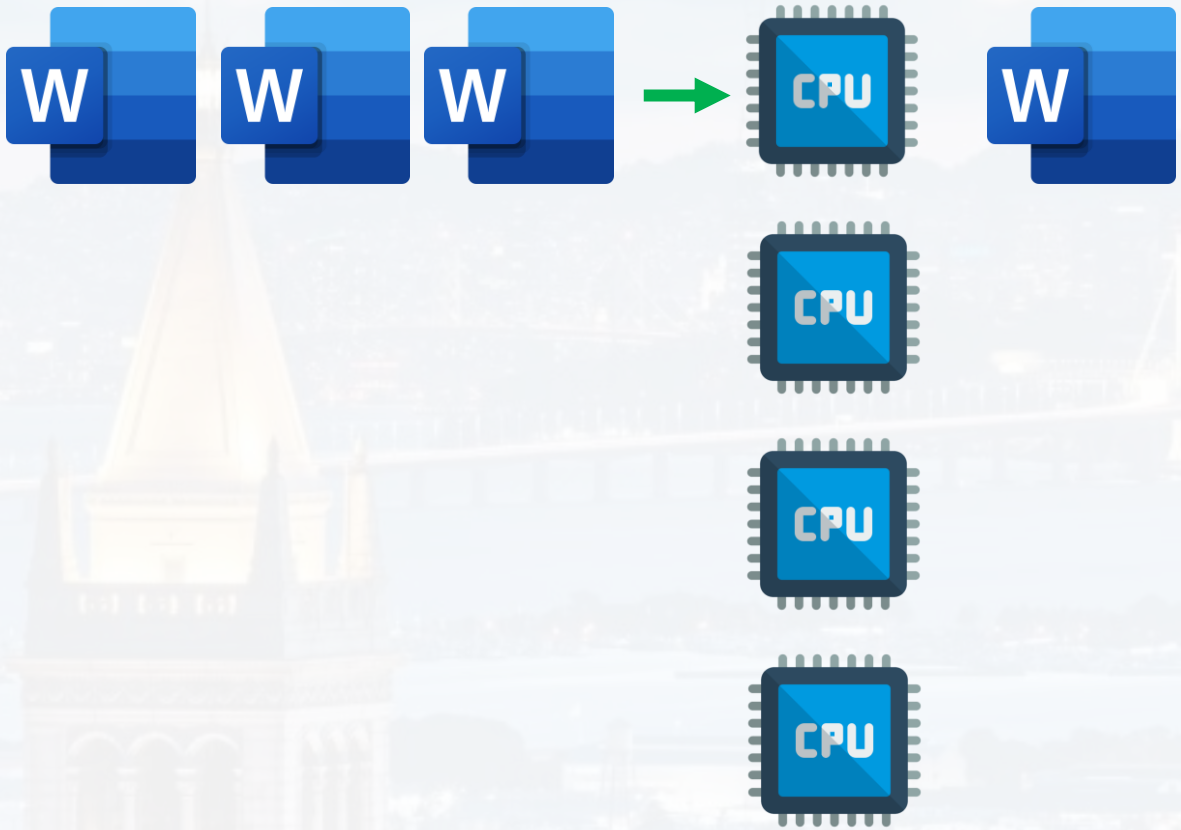
standard:



Parallel & CUDA:

even for moderate models and data: → computational limits

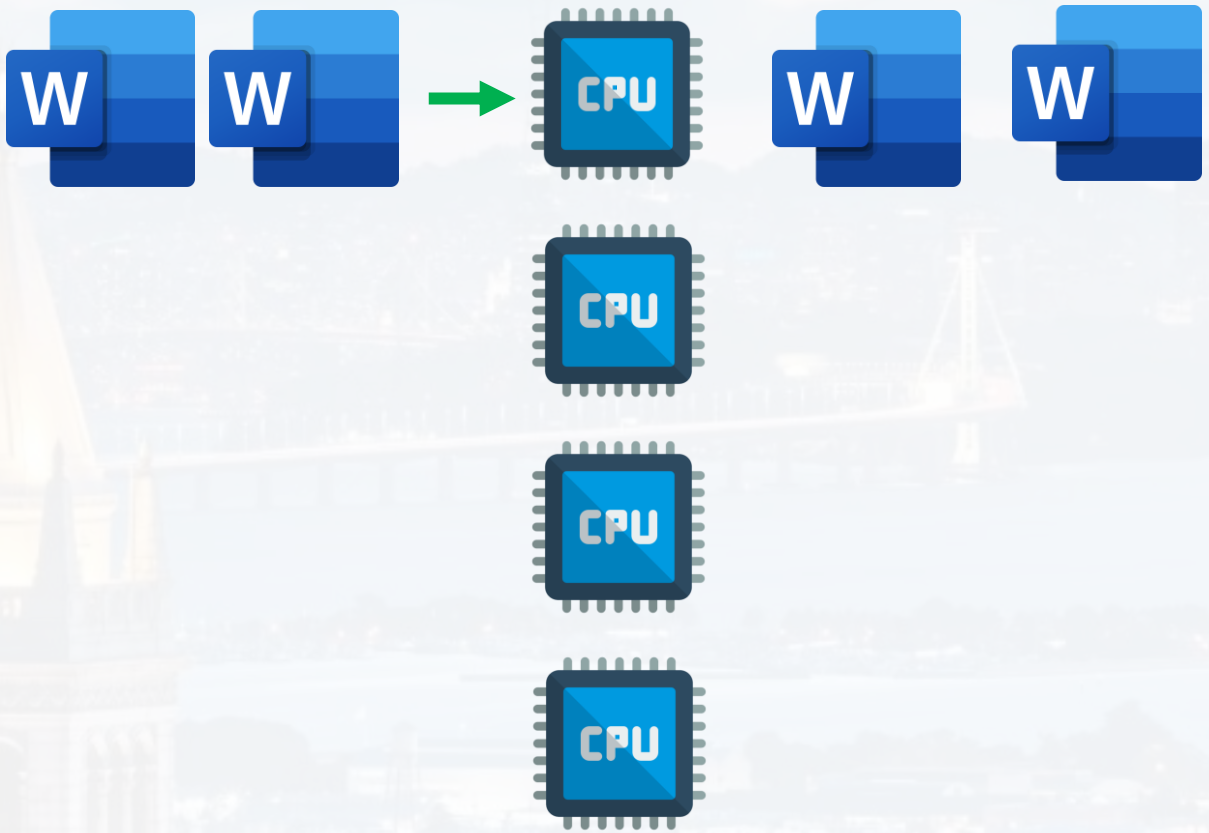
standard:



Parallel & CUDA:

even for moderate models and data: → computational limits

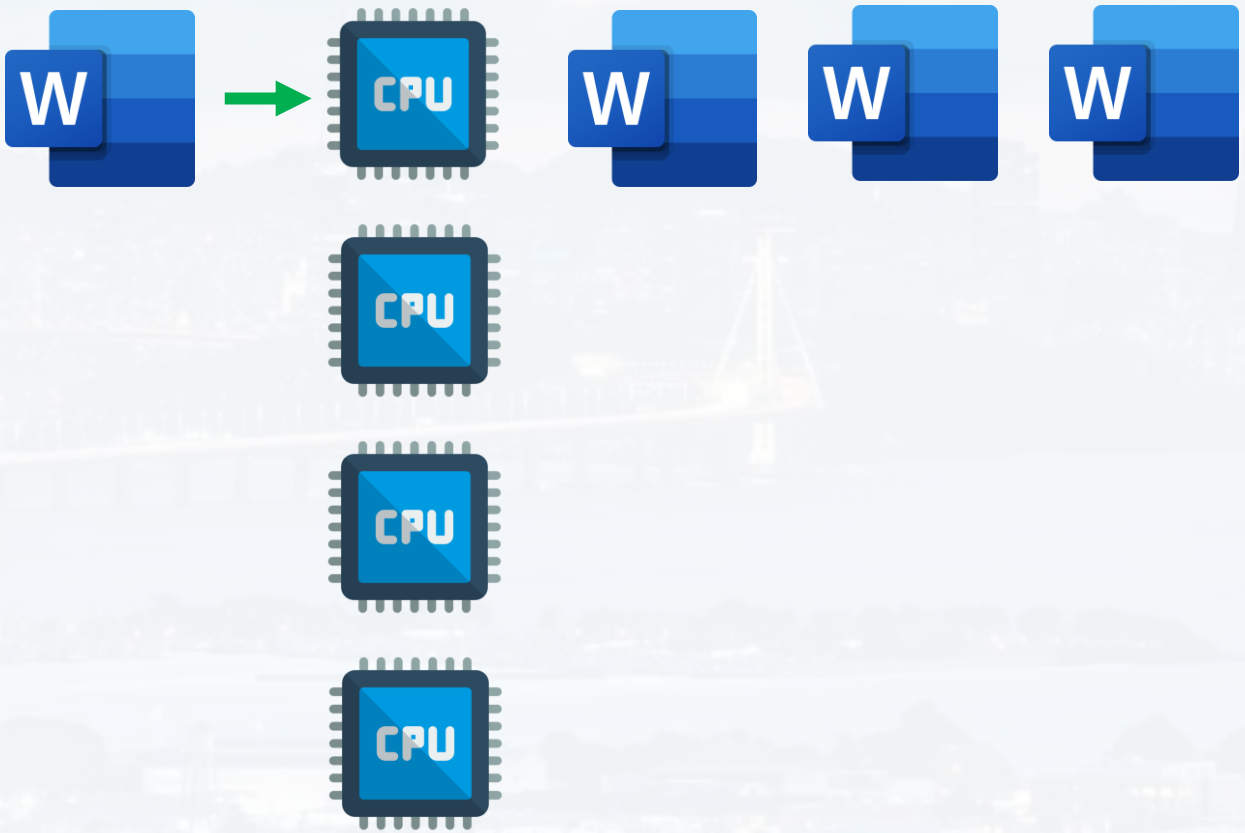
standard:



Parallel & CUDA:

even for moderate models and data: → computational limits

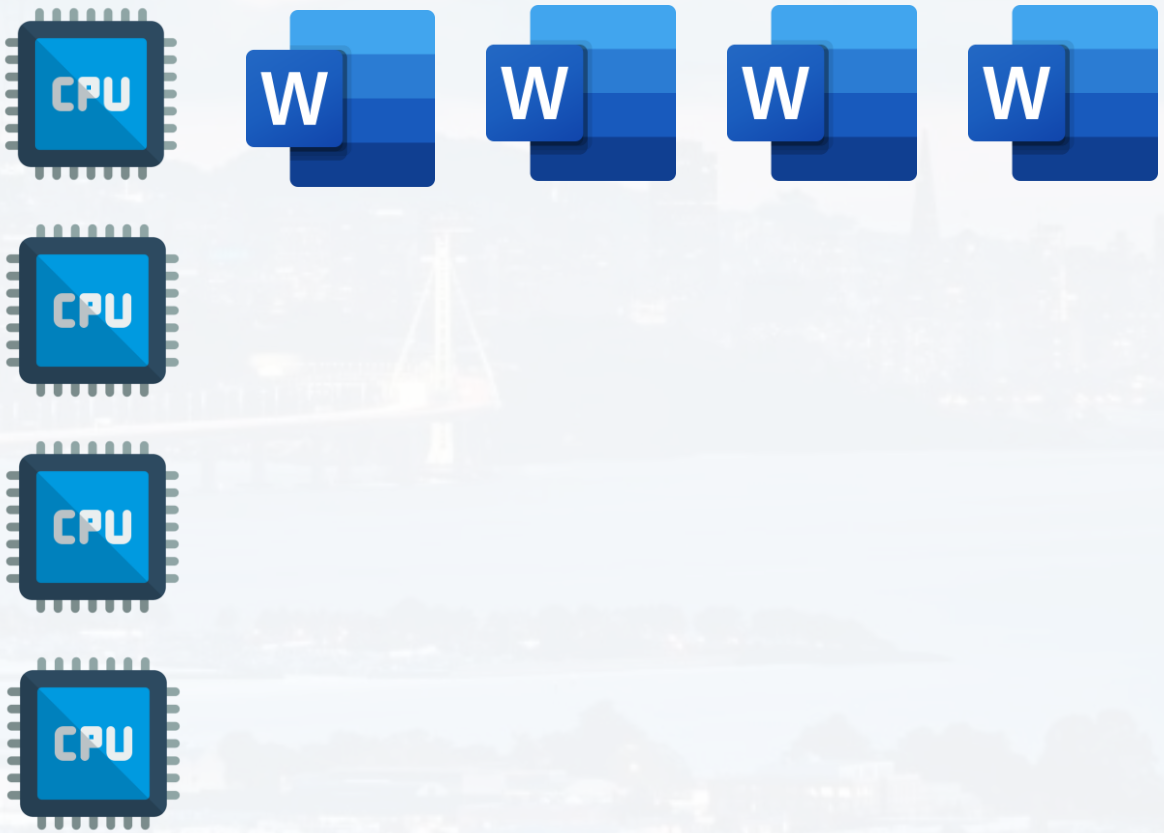
standard:



Parallel & CUDA:

even for moderate models and data: → computational limits

standard:

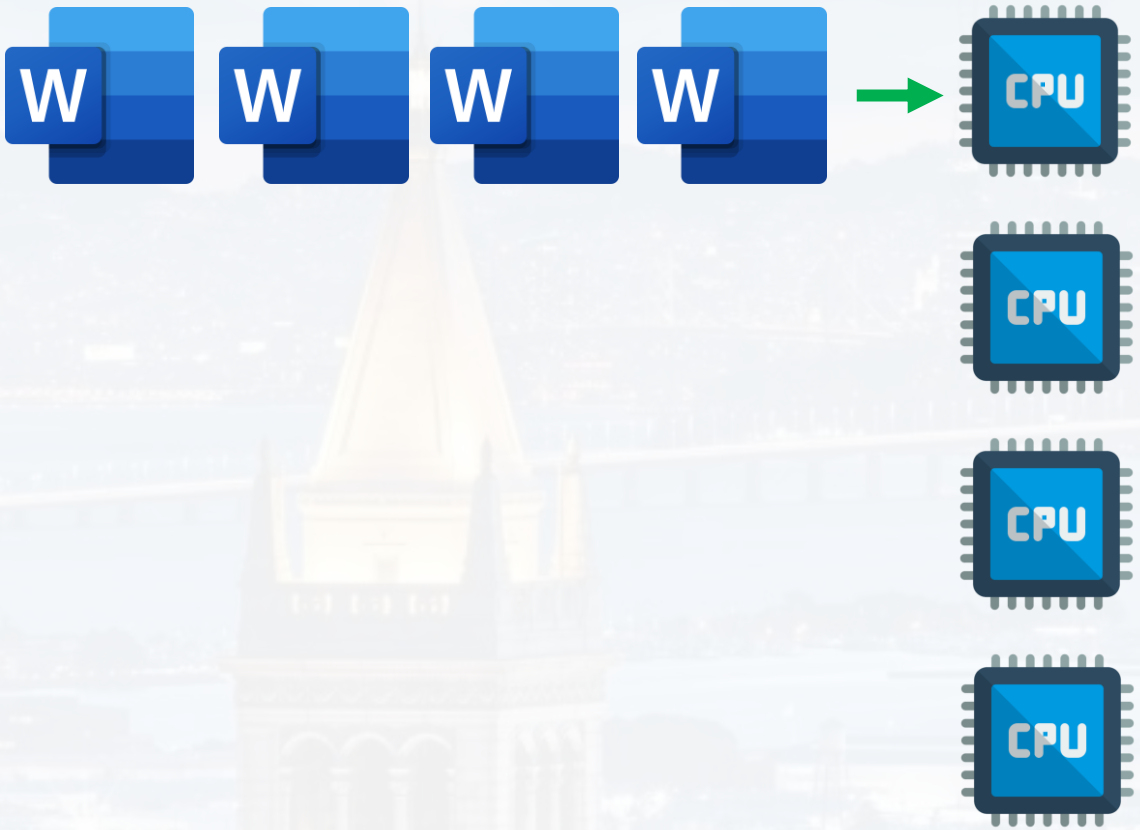


Parallel & CUDA:

even for moderate models and data: → computational limits

same program, but different instances:

- eg. reading different files, and perform the same analysis
- frequency grid search (different sub intervals)

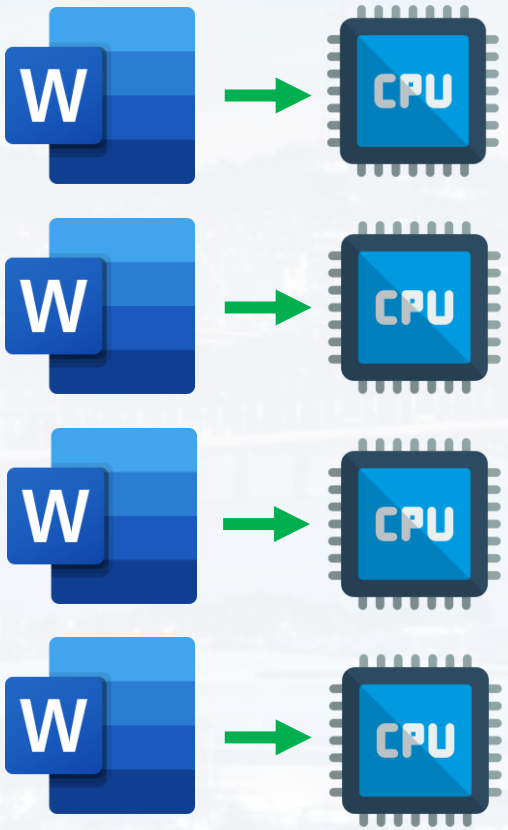


Parallel & CUDA:

even for moderate models and data: → computational limits

same program, but different instances:

- eg. reading different files, and perform the same analysis
- frequency grid search (different sub intervals)

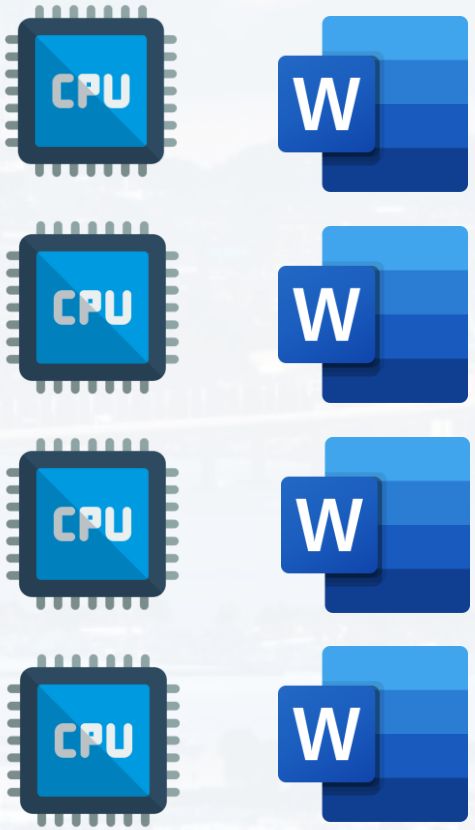


Parallel & CUDA:

even for moderate models and data: → computational limits

same program, but different instances:

- eg. reading different files, and perform the same analysis
- frequency grid search (different sub intervals)



Parallel & CUDA:

even for moderate models and data: → computational limits

same program, but different instances:

- eg. reading different files, and perform the same analysis
- frequency grid search (different sub intervals)

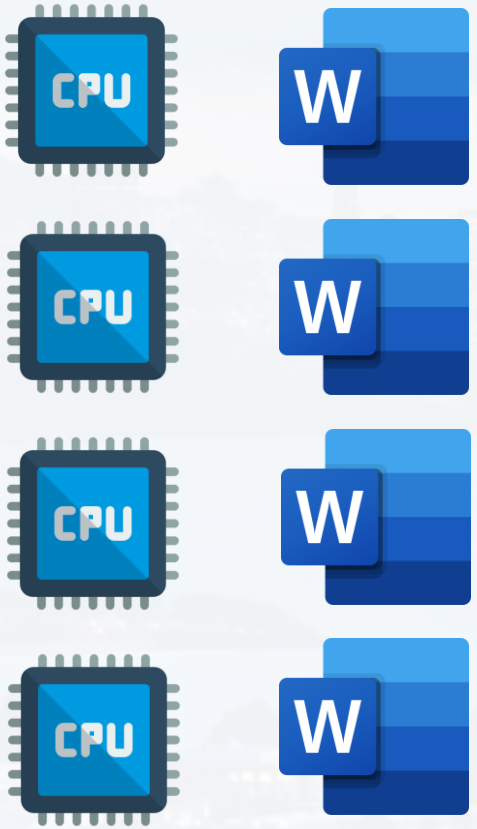
```
pip install multiprocess

import multiprocessing as mp

Processes = [mp.Process(target = MyFun, args = (List,))\
             for file in list_filenames]
```

```
for p in Processes:
        p.start()
```

allocating processes to CPUs

```
for p in Processes:
        p.join()
```

running and finishing processes

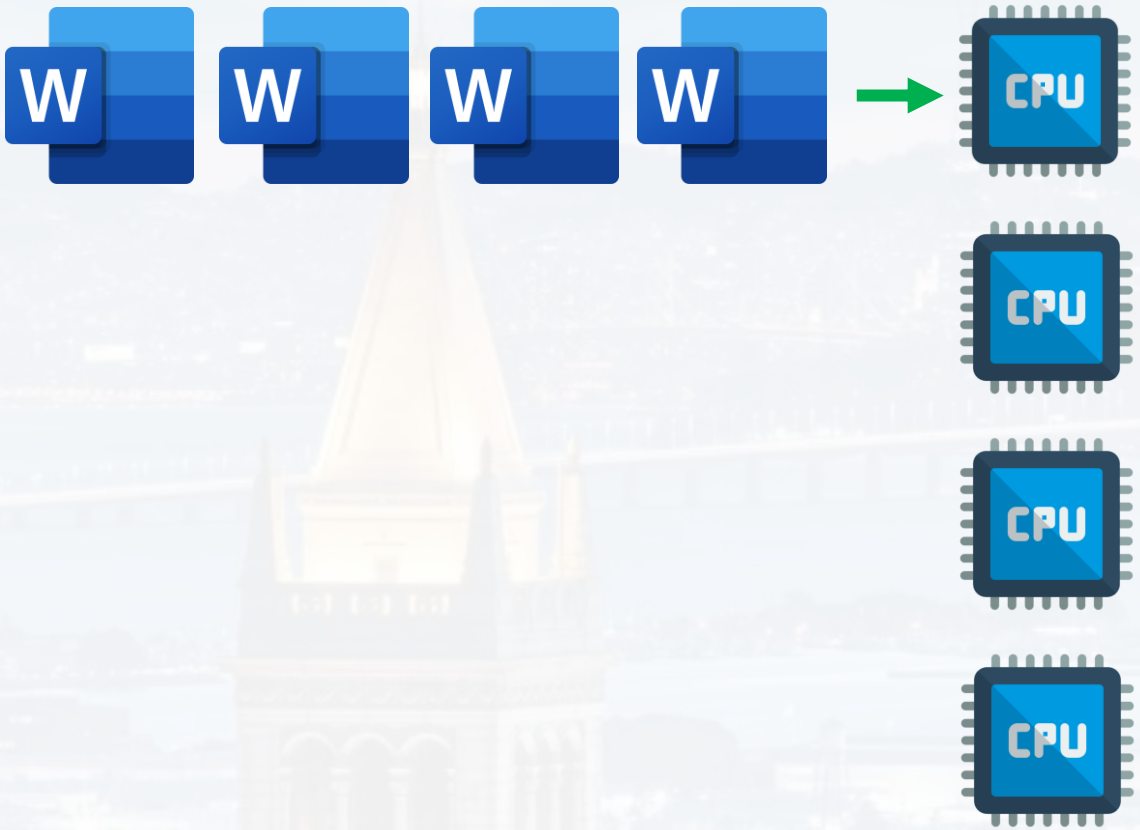


Parallel & CUDA:

even for moderate models and data: → computational limits

parallelizing the process itself:

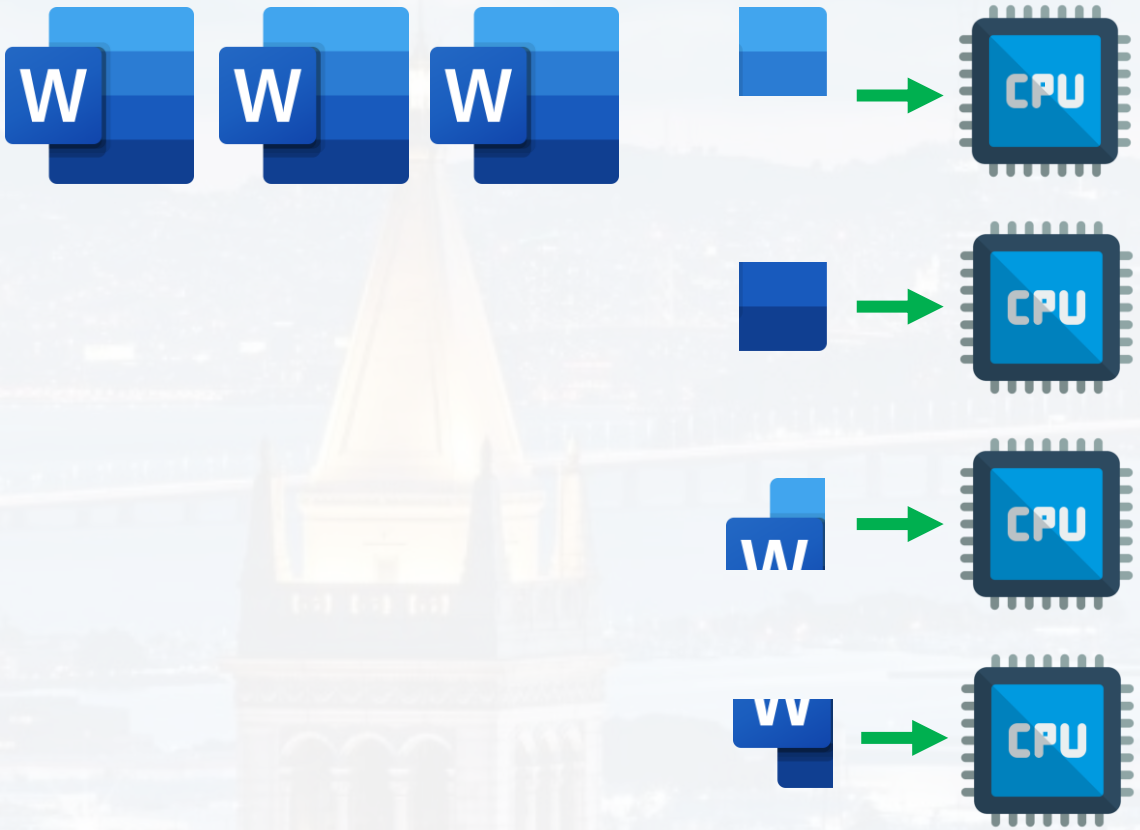
- eg. a non vectorized function in a for loop



Parallel & CUDA:

even for moderate models and data: → computational limits

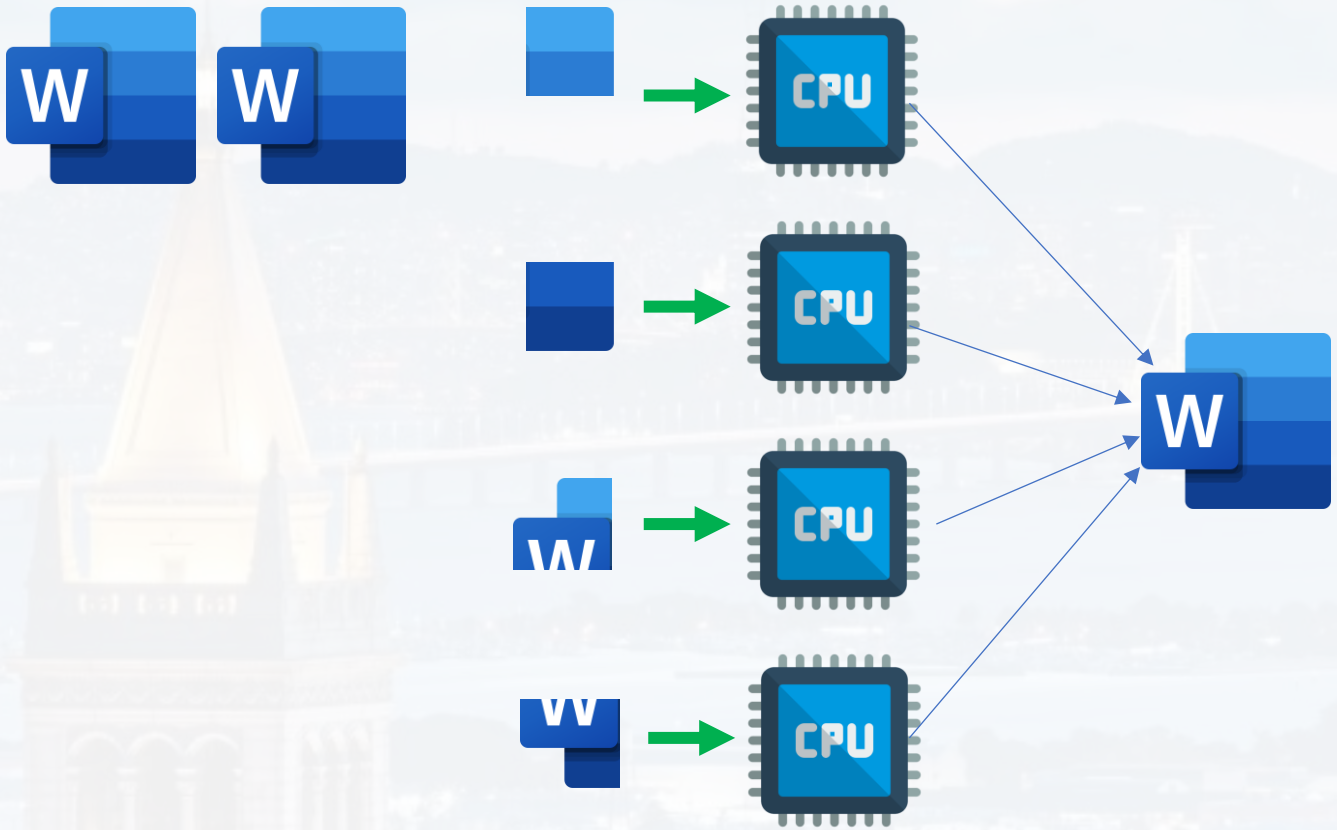
parallelizing the process itself: - eg. a non vectorized function in a for loop



Parallel & CUDA:

even for moderate models and data: → computational limits

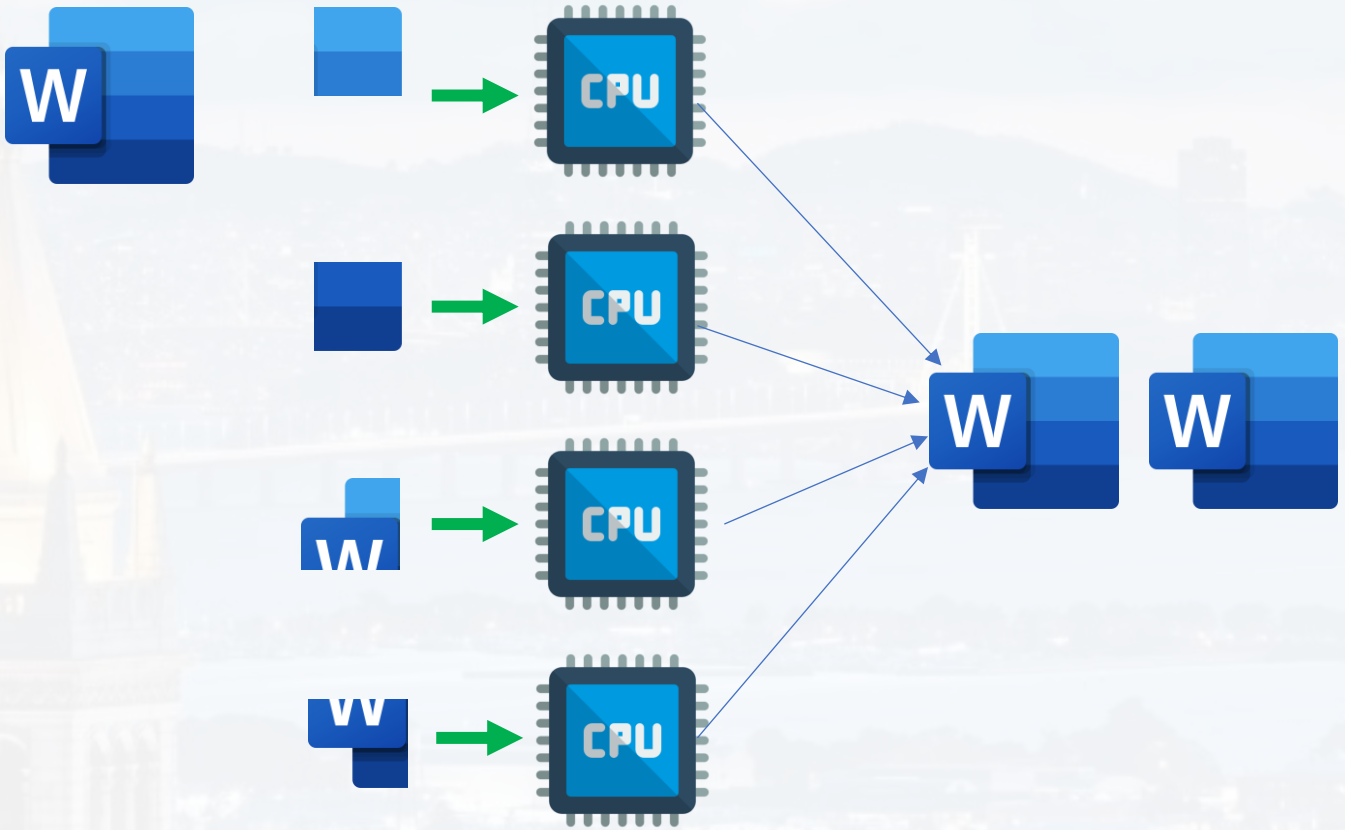
parallelizing the process itself: - eg. a non vectorized function in a for loop



Parallel & CUDA:

even for moderate models and data: → computational limits

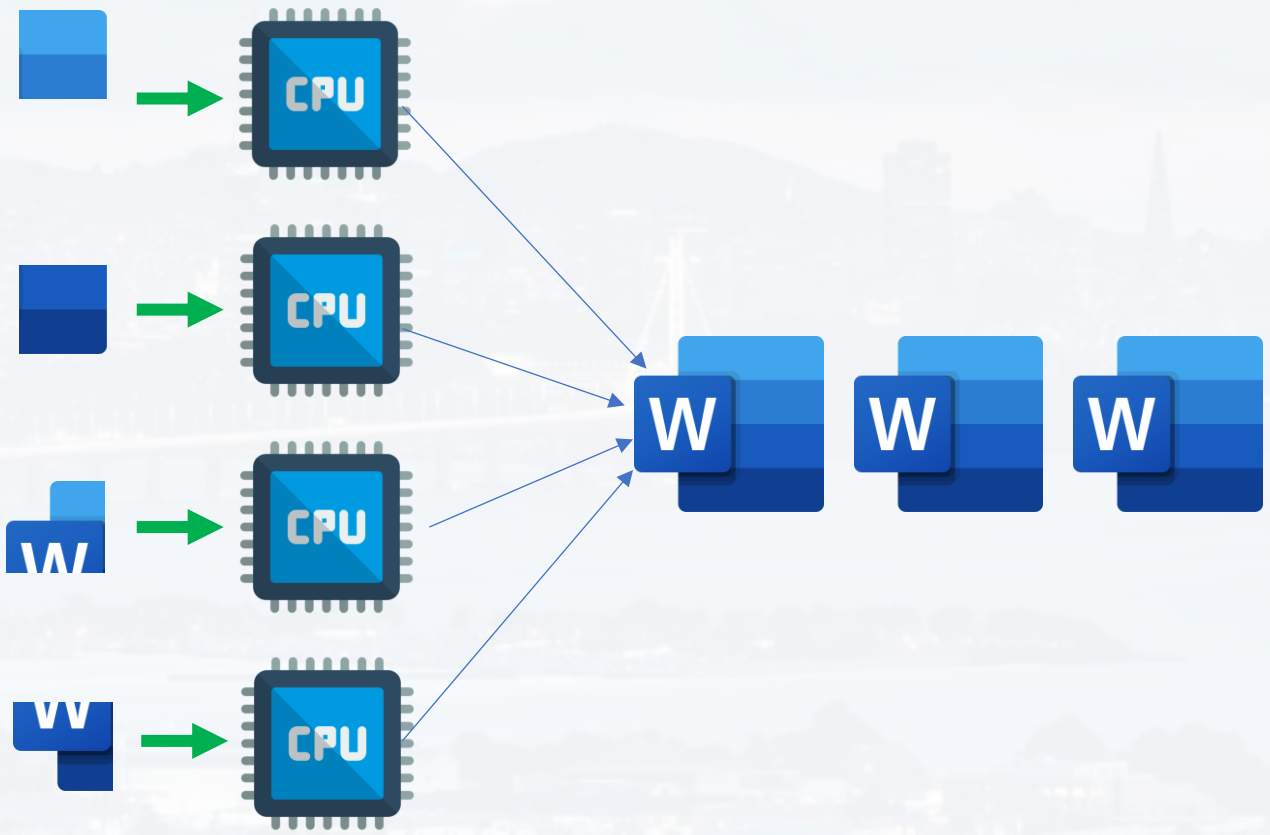
parallelizing the process itself: - eg. a non vectorized function in a for loop



Parallel & CUDA:

even for moderate models and data: → computational limits

parallelizing the process itself: - eg. a non vectorized function in a for loop



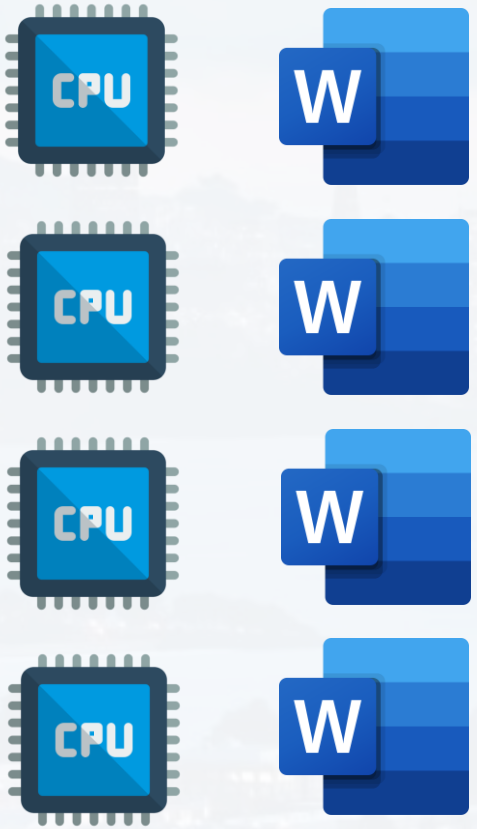
Parallel & CUDA:

even for moderate models and data: → computational limits

parallelizing the process itself: - eg. a non vectorized function in a for loop

```
import multiprocessing as mp


with mp.Pool(processes = len(list_filenames)) as pool:
        All = pool.map(MyFun, list_filenames)

return All
```

combined output

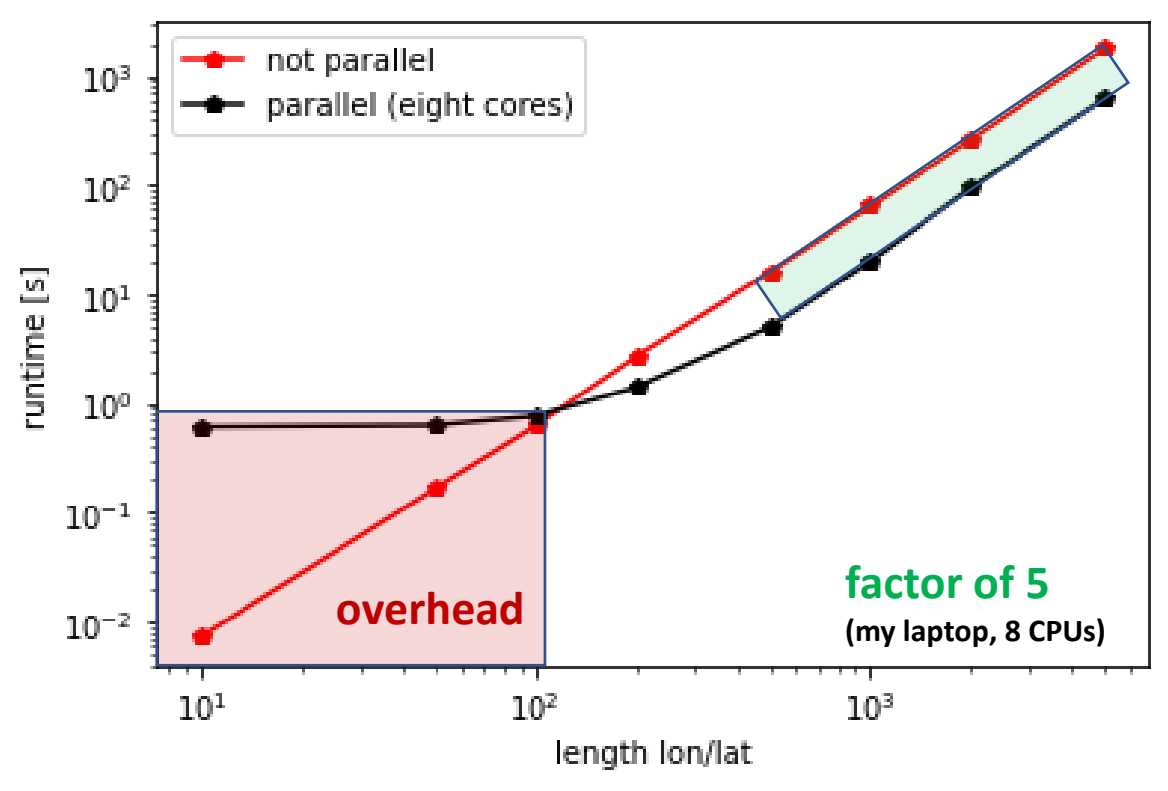


Parallel & CUDA:

parallelizing the process itself: - eg. a non vectorized function in a for loop

same program, but different instances:
- eg. reading different files, and perform the same analysis
- frequency grid search (different sub intervals)

note: preparing and coordinating the different processes takes time → **overhead**



parallelizing two nested for loops

Machine Learning Algorithms:

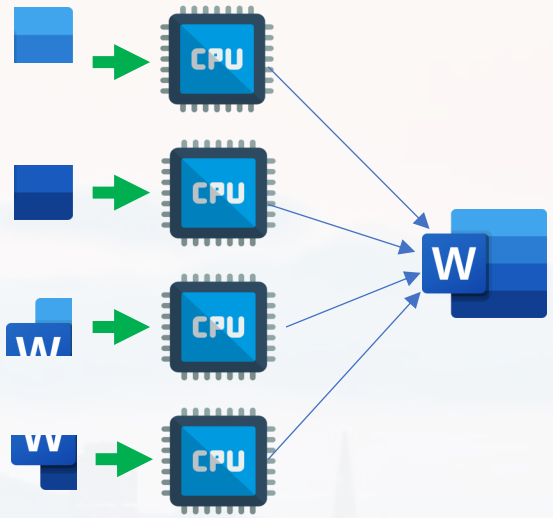
"What was initially thought to be a simple process is in fact an incredibly complicated, intricate, and complex system that I've codified and organized into a few easy-to-follow rules that are more difficult to implement than you'd think."

Outline

- Parallel Processing
- **Using Map**
- Using Process
- CUDA & PyTorch

Parallel & CUDA:

example1: reading excel files, extracting some content and combing it.




```
import multiprocessing as mp
```

a) writing a function that does one thing at the time

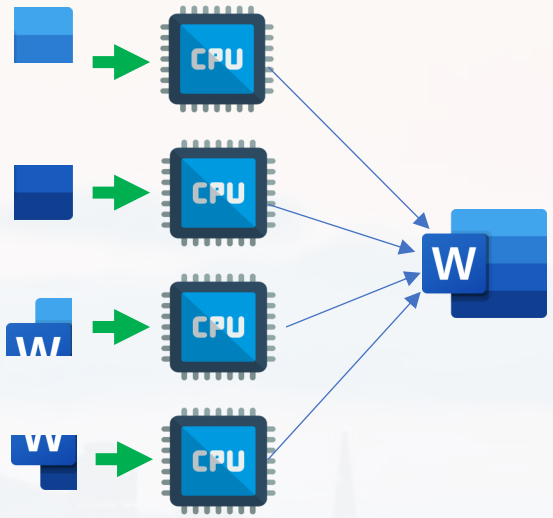
```
def MyFun(filename: str) -> np.array:
    #reads huge data set
    data = pd.read_excel(filename)
    return np.array(data.time)
```

b) writing a **main** function that calls the first function using **map**

```
def Parallel(self, list_filenames: list) -> list:

        with mp.Pool(processes = cpu_count() - 1) as pool:
                All = pool.map(MyFun, list_filenames)

        return All
```

Parallel & CUDA:

example1: reading excel files, extracting some content and combing it.



check out the function Map.py

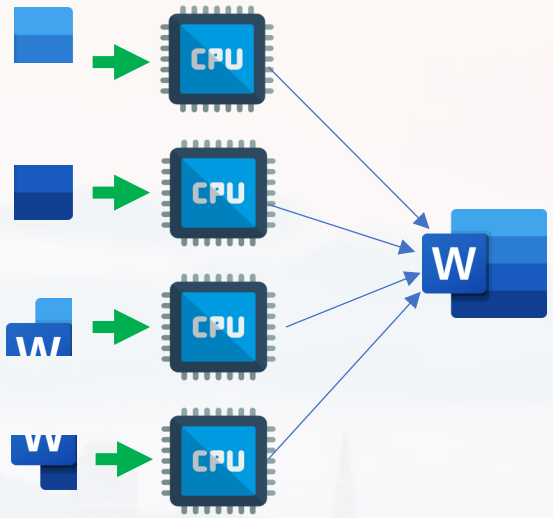
```
from Map import *

globals()['MyFun'] = my_timer(MyFun)

MAP                = Parallel_MAP()
```

```
In [7]: out  = MyFun(list_filenames[0])
Total runtime: 26.93699999997625 seconds

In [8]: out1 = MAP.Serial(list_filenames)
Total runtime: 151.76500000001397 seconds

In [9]: out2 = MAP.Parallel(list_filenames)
Total runtime: 49.14100000000326 seconds
```

Parallel & CUDA:

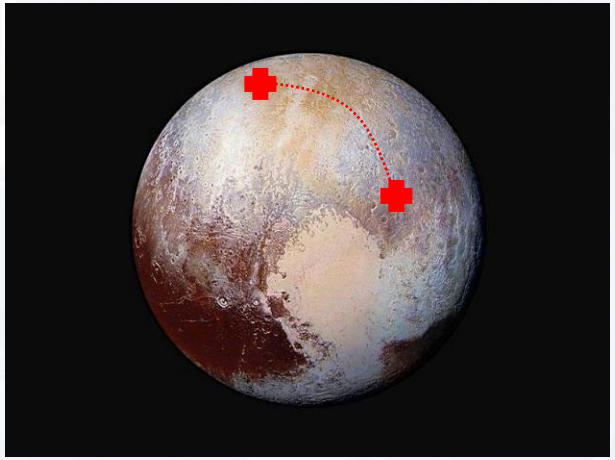
<u>example2: parallelizing a non vectorized function.</u>

geodesic from GeoPy can take only **one pair of coordinates** at the time




```
coords1 = (lon[i],lat[i])
coords2 = (lon[j],lat[j])

d       = geodesic(coords1, coords2).m

distance[i,j] = d
distance[j,i] = d
```

an example of a non vectorized method

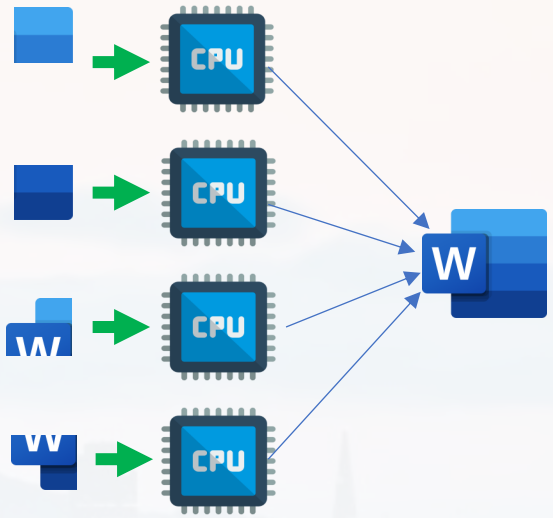


$$d^2s = R^2dv^2 + R^2sin^2v\, d\phi^2$$

Parallel & CUDA:

<u>example2: parallelizing a non vectorized function.</u>

a) writing a function that does one thing at the time

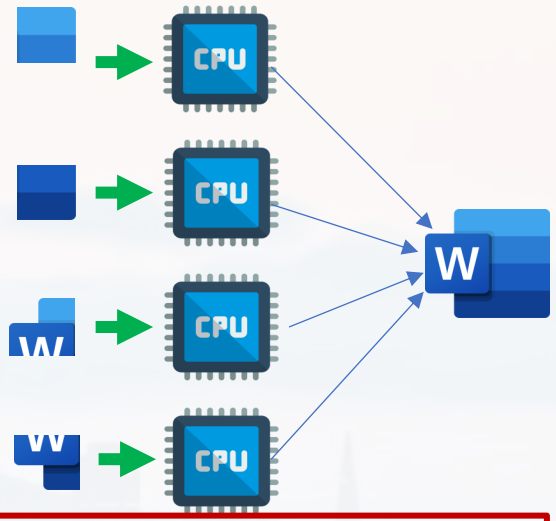



```python
from geopy.distance import geodesic

def calculate_distance(args):

    i, j, lat, lon = args

    coords1 = (lon[i], lat[i])
    coords2 = (lon[j], lat[j])

    d = geodesic(coords1, coords2).m

    return i, j, d
```

Parallel & CUDA:

example2: parallelizing a non vectorized function.

b) writing a **main** function that calls the first function using **map**




```
import numpy as np
from multiprocessing import Pool
from calculate_distance import calculate_distance

def DistanceGeoParallel(lat, lon):

        n        = len(lat)
        distance = np.zeros((n, n))

        args_list = [(i, j, lat, lon) for i in range(n) for j in range(i+1, n)]

        with Pool() as pool:
            results = pool.map(calculate_distance, args_list)

        for i, j, d in results:
            distance[i, j] = d
            distance[j, i] = d

        return distance
```

calling subroutine

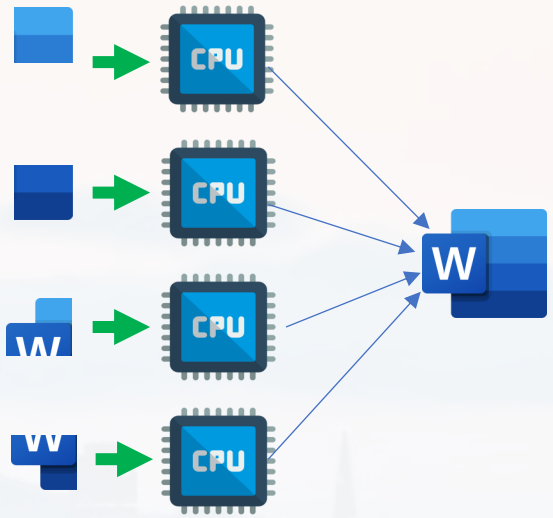
creating list of all variables and indices **(be careful: j starts at i+1 for symmetry reasons!)**

actual parallelization

Parallel & CUDA:

check out:

```
from multiprocess import cpu_count
cpu_count()
```



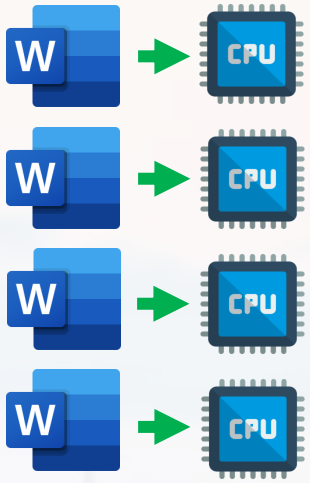
- calling CPUs via `Pool` **itself takes time** and creates **overhead**
  → only effcient for larger runtimes
- processes have to be **independent** *(i.e. calculating distances, linear algebra, see also AI)*
- functions have to be **independent** *(function A **is not** a subroutine of B)*
- processes should not contain `lambda` or `map`

"What was initially thought to be a simple process is in fact an incredibly complicated, intricate, and complex system that I've codified and organized into a few easy-to-follow rules that are more difficult to implement than you'd think."

Outline

- Parallel Processing
- Using Map
- **Using Process**
- CUDA & PyTorch

Parallel & CUDA:



example: reading excel files and saving them as csv.

```
import multiprocessing as mp
```

a) writing a function that does one thing at the time

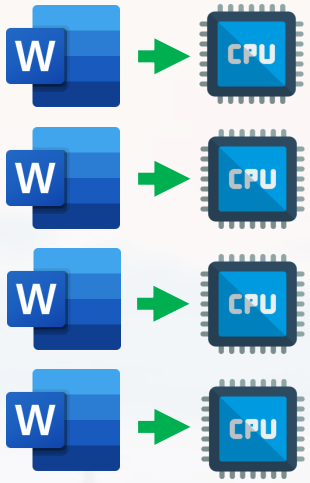
```
def MyFun(filename: str):
    #reads & saves huge data set
    data = pd.read_excel(filename)
    data.to_csv('Data_set_' + str(time.monotonic()).replace('.','-') + '.csv'))
```

adding a time stamp to the output file name

b) writing a **main** function that calls the first function using Process

Parallel & CUDA:

<u>example: reading excel files and saving them as csv.</u>



b) writing a **main** function that calls the first function using Process

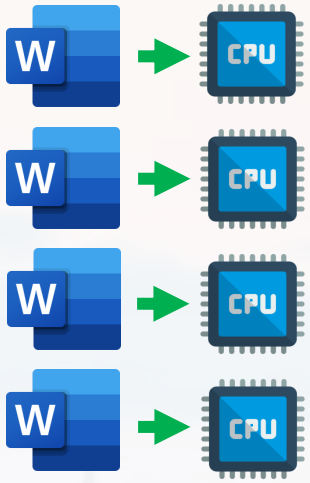
```
def Parallel(self, list_filenames: list) -> list:

        Processes = [mp.Process(target = MyFun, args = (file,))\
                     for file in list_filenames]

        for p in Processes:
            p.start()

        for p in Processes:
            p.join()
```

allocating processes to CPUs

running and finishing processes

Parallel & CUDA:

Process



example: reading excel files and saving them as csv.

check out the function `Process.py`

```
from Process import *

globals()['MyFun'] = my_timer(MyFun)

Pro                    = Parallel_Process()
```

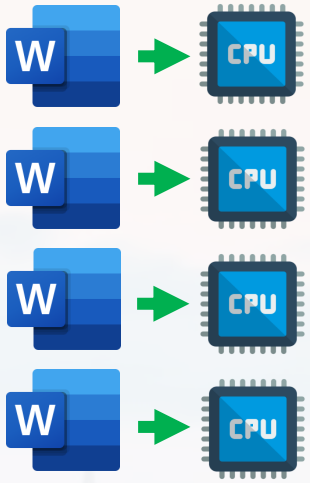
```
out  = MyFun(list_filenames[0])
Pro.Serial(list_filenames)
Pro.Parallel(list_filenames)
```

```
Total runtime: 27.7030000003772 seconds
Total runtime: 172.0309999995902 seconds
Total runtime: 57.4070000000652 seconds
```

Parallel & CUDA:

Process

example: reading excel files and saving them as csv.



Data_set_461714-39.csv	20/11/2024 02:57	
Data_set_461742-39.csv	20/11/2024 02:58	
Data_set_461771-984.csv	20/11/2024 02:58	
Data_set_461800-75.csv	20/11/2024 02:59	
Data_set_461829-625.csv	20/11/2024 02:59	
Data_set_461858-187.csv	20/11/2024 03:00	

every 30 seconds

Data_set_461912-031.csv	20/11/2024 03:00	
Data_set_461912-39.csv	20/11/2024 03:00	
Data_set_461912-765.csv	20/11/2024 03:00	
Data_set_461912-828.csv	20/11/2024 03:00	
Data_set_461913-281.csv	20/11/2024 03:00	
Data_set_461914-14.csv	20/11/2024 03:00	

every 0.3 seconds

```
out  = MyFun(list_filenames[0])
Pro.Serial(list_filenames)
Pro.Parallel(list_filenames)
```

```
Total runtime: 27.70300000003772 seconds
Total runtime: 172.03099999995902 seconds
Total runtime: 57.40700000000652 seconds
```

"What was initially thought to be a simple process is in fact an incredibly complicated, intricate, and complex system that I've codified and organized into a few easy-to-follow rules that are more difficult to implement than you'd think."

Outline

- Parallel Processing
- Using Map
- Using Process
- **CUDA & PyTorch**

Parallel & CUDA:

problem: even for moderate setups, computational time becomes the limiting factor

self made ANN → keras/tensor flow → pytorch → pytorch on GPU (Cuda)

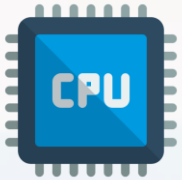
speed

```
512/512 [==============================] - 4072s 8s/step - loss: 3.4534 - accuracy: 0.3338
Epoch 4/5
512/512 [==============================] - ETA: 0s - loss: 3.1926 - accuracy: 0.3980     saved  ../data/segmentation
pics/checkpoints//.3
512/512 [==============================] - 3363s 7s/step - loss: 3.1926 - accuracy: 0.3980
Epoch 5/5
512/512 [==============================] - ETA: 0s - loss: 3.0071 - accuracy: 0.4318   saved  ../data/segmentation
pics/checkpoints//.4
512/512 [==============================] - 3616s 7s/step - loss: 3.0071 - accuracy: 0.4318
```

Lenovo T14, NVIDIA GeForce MX450: (simple LSTM)

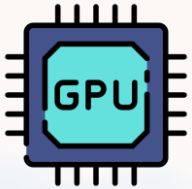
Keras (CPU):	**300 sec**
PyTorch (CPU):	11 sec
PyTorch (GPU):	**3 sec**

Parallel & CUDA:



training AI

→ mainly matrix operations
→ GPUs are a lot better at it!



NVIDIA CUDA

CUDA is the link of your GPU to Python (PyTorch)

check, if graphic card is on list

check your graphics device:

→ Windows command shell prompt or **environment command shell**

→ type `nvidia-smi`

→ press *Enter*

```
(base) PS C:\Users\MMH_user> nvidia-smi
Tue Mar 17 20:36:51 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 573.91                 Driver Version: 573.91         CUDA Version: 12.8     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                  Driver-Model | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce MX450           WDDM   |   00000000:01:00.0 Off |                  N/A |
| N/A    [illegible]    P0            N/A  /    9W |       0MiB /   2048MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Parallel & CUDA:

1) set up CUDA environment
2) install libraries
3) check if it worked
4) test in your UI

Watch (material module 13)

Install_Cuda.mp4
A Cuda_Intro.mp4
B Cuda_Installation.mp4
C Cuda_CodeModification.mp4

Parallel & CUDA:

1) **set up CUDA environment**
2) install libraries
3) check if it worked
4) test in your UI

It makes sense to set up a **specific environment** in order to install **specific versions** of the required packages!

```
conda create -n MyCUDAenv python=3.11
```

```
(base) PS C:\Users\MMH_user> conda create -n MyCUDAenv python=3.11
```

don't install the most recent version in order to avoid version conflicts!

```
conda activate MyCUDAenv
```

```
(base) PS C:\Users\MMH_user> conda activate MyCUDAenv
```

```
(MyCUDAenv) PS C:\Users\MMH_user>
```

Parallel & CUDA:

It makes sense to set up a **specific environment** in order to install **specific versions** of the required packages!

1) set up CUDA environment
2) **install libraries**
3) check if it worked
4) test in your UI

```
conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia
```

Even if my GPU could run on cuda 12.8, we install an older version in order to avoid version conflicts!

```
(MyCUDAenv) PS C:\Users\MMH_user> conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia
```

Parallel & CUDA:

We check if it all worked!

1) set up CUDA environment
2) install libraries
3) **check if it worked**
4) test in your UI

```
python -c "import torch; print(torch.cuda.is_available()); print(torch.version.cuda)"
```

```
(MyCUDAenv) PS C:\Users\MMH_user> python -c "import torch; print(torch.cuda.is_available()); print(torch.version.cuda)"
```

```
True
12.1
```

```
python -c "import torch; print(torch.cuda.get_device_name(0))"
```

```
(MyCUDAenv) PS C:\Users\MMH_user> python -c "import torch; print(torch.cuda.get_device_name(0))"
```

```
NVIDIA GeForce MX450
```

PyTorch and CUDA

1) set up CUDA environment
2) install libraries
3) check if it worked
4) **test in your UI**

We check if it all worked!

→ Install and open your favorite UI

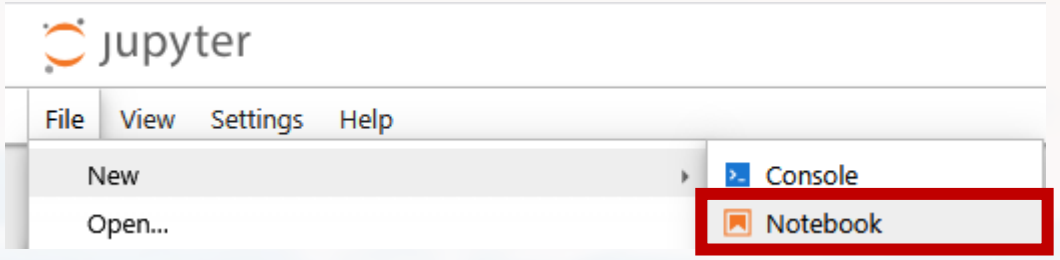
CheckMyCuda.py

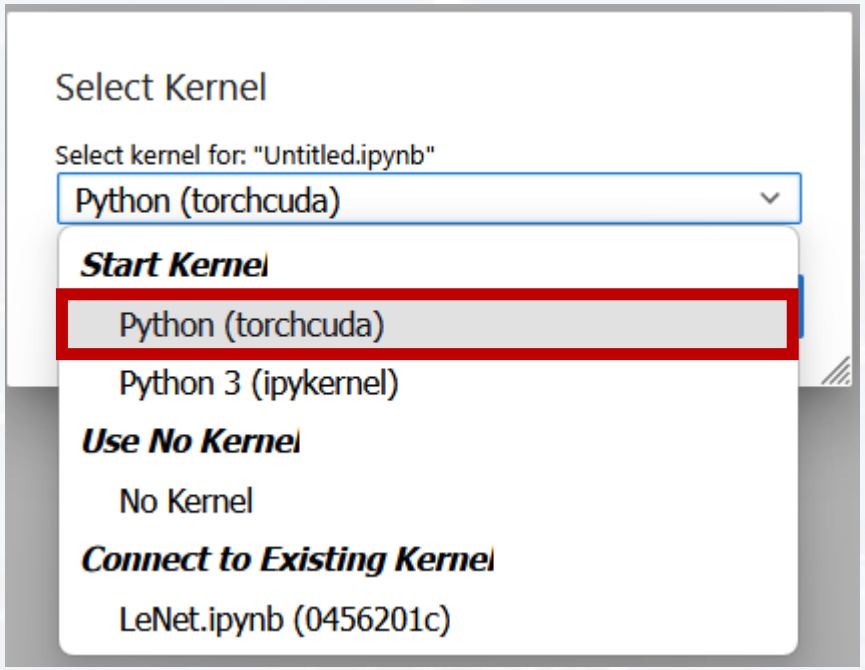
```python
import torch

def test_cuda():
    print("PyTorch version: ", torch.__version__)
    print("CUDA version: ", torch.version.cuda)
    print("CUDA Available: ", torch.cuda.is_available())
    if torch.cuda.is_available():
        print("Number of GPUs: ", torch.cuda.device_count())
        print("GPU Name: ", torch.cuda.get_device_name(0))

if __name__ == "__main__":
    test_cuda()
```

```
PyTorch version:  2.5.1
CUDA version:  12.1
CUDA Available:  True
Number of GPUs:  1
GPU Name:  NVIDIA GeForce MX450
```

Parallel & CUDA:

1) set up CUDA environment
2) install libraries
3) check if it worked
4) **test in your UI**






```python
import torch

def test_cuda():
    print("PyTorch version: ", torch.__version__)
    print("CUDA version: ", torch.version.cuda)
    print("CUDA Available: ", torch.cuda.is_available())
    if torch.cuda.is_available():
        print("Number of GPUs: ", torch.cuda.device_count())
        print("GPU Name: ", torch.cuda.get_device_name(0))

if __name__ == "__main__":
    test_cuda()
```

```
PyTorch version:  2.5.1
CUDA version:  12.1
CUDA Available:  True
Number of GPUs:  1
GPU Name:  NVIDIA GeForce MX450
```

Parallel & CUDA:

The key part in PyTorch is to set all matrices and the model **to the device (CPU or GPU)**

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
In [13]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    ...: print("Using device:", device)
Using device: cuda
```

Congratulation! If you see this, you are ready to go!

Parallel & CUDA:

The key part in PyTorch is to set all matrices and the model **to the device (CPU or GPU)**

```
In [13]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    ...: print("Using device:", device)
Using device: cuda
```

Torch objects like **`model`** or **`torch.tensor`** have the property **`.to`**

```
TrainX = torch.tensor(TrainX, dtype = torch.float32)
TrainY = torch.tensor(TrainY, dtype = torch.float32)
```

turning numpy `array` into `torch.tensor`

```
TrainX = TrainX.to(device)
TrainY = TrainY.to(device)
```

```
model  = model.to(device)
```

allocating objects to the device

Parallel & CUDA:

The key part in PyTorch is to set all matrices and the model **to the device (CPU or GPU)**

```
In [13]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    ...: print("Using device:", device)
Using device: cuda
```

When running the training, we need to **synchronize** between GPU (for training the model) and CPU (for everything else)…

```
torch.cuda.synchronize()

        #training loop
        for epoch in range(n_epochs):
                outputs = model(TrainX)

                ... #do stuff...

torch.cuda.synchronize()
```

Parallel & CUDA:

The key part in PyTorch is to set all matrices and the model **to the device (CPU or GPU)**

```
In [13]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    ...: print("Using device:", device)
Using device: cuda
```

When running the training, we need to **synchronize** between GPU (for training the model) and CPU (for everything else)…

…and later detach the model from the GPU

```
PredY = model(TestX).detach().to('cpu').numpy()
```

Parallel & CUDA:

check out script **HowToRun.py**

that runs the **same LSTM model** for Keras and PyTorch

```
n_neurons  = 100
n_epochs   = 200
dt_past    = 30
dt_futu    = 10
n_features = 1
n_sample   = 1

n_stack    = 3
```

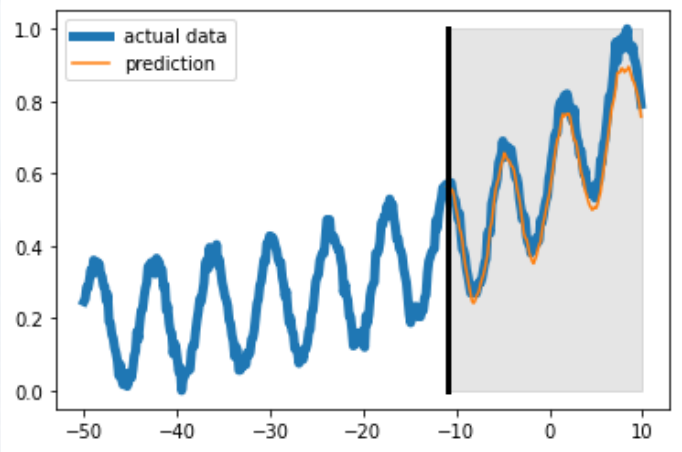
number of stacked LSTM

Parallel & CUDA:

check out script **HowToRun.py**

that runs the **same LSTM model** for Keras and PyTorch

```
n_neurons  = 100
n_epochs   = 200
dt_past    = 30
dt_futu    = 10
n_features = 1
n_sample   = 1

n_stack    = 3
```



Lenovo T14, NVIDIA GeForce MX450:

Keras LSTM (CPU):	**300 sec**
PyTorch (CPU):	11 sec
PyTorch (GPU):	**3 sec**

Thank you very much for your attention!